

Audit-Grade RAG for Engineering Standards

Local Installation and Operations Guide — Windows 11 / Quadro RTX 5000

Single-workstation deployment. Worked example: EASA CS-25 Amendment 27

17 September 2026 — version 2.0

Contents

1 Purpose and scope	4
1.1 What this system must do	4
1.2 Scope and companion archive	4
1.3 Target host	4
1.4 How to read this	5
2 Component selection	6
2.1 The stack	6
2.2 Generation: Ministral 3 14B Instruct 2512	6
2.3 Embedding: BGE-M3	7
2.4 Reranking: bge-reranker-v2-m3	7
2.5 Vector store: Qdrant	8
2.6 Parsing: PyMuPDF	8
2.7 One constraint that is not technical	8
3 Architecture	9
3.1 The pipeline	9
3.2 Why the LLM appears only once	9
3.3 The three components that make this auditable	10
3.4 What the architecture does not do	10
4 Layout, ports and directories	11
4.1 Process map	11
4.2 Directory layout	11
5 Host preparation	13
5.1 NVIDIA driver and CUDA runtime	13
5.2 Python	13
5.3 Set HF_HOME before anything downloads	13
5.4 Supporting tools	14
5.5 Docker Desktop and the WSL2 memory cap	14
5.6 Defender exclusions and power plan	14
6 Qdrant	15
6.1 Storage: use a named volume, not a bind mount	15
6.2 Compose file	15
6.3 Start and verify	16
6.4 Note on PowerShell and JSON	16
7 LLM serving	18
7.1 VRAM budget	18
7.2 Install and configure Ollama	18
7.3 Pull the model and build the serving profile	19
8 Retrieval service	20
8.1 PyTorch for Turing	20
8.2 Remaining dependencies	20
8.3 Pre-download the weights	20

8.4 Start and verify	21
9 Ingestion	22
9.1 The corpus register and the page-offset problem	22
9.2 Step 1 — confirm the text layer	23
9.3 Step 2 — validate the parser against the publisher	23
9.4 Step 3 — dry run	24
9.5 Step 4 — ingest	24
9.6 Step 5 — verify coverage	24
9.7 Step 6 — measure what you missed	25
9.8 Step 7 — the manual spot-check	25
10 Adapting the parser to a publisher	26
10.1 The diagnostic loop	26
10.2 What CS-25 required	26
10.3 The generic decimal trap	27
10.4 The collision guard	27
10.5 Sub-paragraph suffixes	27
11 Query pipeline	28
11.1 Hybrid retrieval	28
11.2 The abstention thresholds, explained	28
11.2.1 What a reranker score actually is	28
11.2.2 $\text{TAU_ABSTAIN} = 0.35$ — whether to answer at all	29
11.2.3 $\text{TAU_INCLUDE} = 0.15$ — which passages the model sees	29
11.2.4 A worked example	29
11.2.5 How to calibrate	30
11.2.6 The other two constants	30
11.3 Citation verification	30
11.4 Audit trail	31
12 The answer contract	32
12.1 The eight rules	32
12.2 Why rule 7 is the most important line in the system	33
12.3 Why “not in corpus” matters more than any answer	33
12.4 Editing the contract	34
13 The web UI	35
13.1 What it shows	35
13.2 Configuration	35
14 Running as Windows services	36
14.1 The environment problem	36
14.2 retrieval-svc	36
14.3 rag-api	36
14.4 Reboot test	37
15 Validation	38
15.1 Build a golden set	38
15.2 Run it	38
15.3 Acceptance thresholds	39
15.4 Tuning order	39
16 Operations	40
16.1 Backup	40
16.2 Adding or superseding a standard	40
16.3 Versioning and re-index policy	41
16.4 Health checks	41
17 Troubleshooting	42

18	Appendix A — Version pin table	44
19	Appendix B — Performance on this hardware	45
20	Appendix C — Licence and IP register	46
21	Appendix D — Script reference	47
22	Appendix E — Install checklist	48
23	Appendix F — Sources consulted	49

Chapter 1

Purpose and scope

1.1 What this system must do

An engineer asks a natural-language question about a design, test or qualification requirement. The system returns an answer in which **every assertion is traceable to a retrieved passage**, cited as document identifier, clause number and page. When the corpus does not contain the answer, the system says so.

That last sentence is the hard part. A retrieval-augmented generation stack is easy to stand up and easy to make *plausible*. Making it abstain correctly, and making its citations survive an auditor checking them against the paper standard, requires specific choices in parsing, chunking, retrieval and post-generation validation. This document specifies those choices and gives the commands to install them.

1.2 Scope and companion archive

This version contains no Python listings. Extracting code from a PDF destroys the indentation, which is fatal in Python. All scripts ship in the companion archive `rag-scripts.zip`, whose contents mirror the install layout in §4.2 and are summarised in Appendix D. PowerShell commands remain inline, since those you will type.

The guide is written for any standards corpus. EASA CS-25 Amendment 27 runs through it as the worked example, because it exercises every difficult case: alphanumeric clause numbering, a consolidated two-book structure, appendices with their own numbering scheme, small-capped headings, and a contents section several hundred entries long.

1.3 Target host

Item	Specification	Consequence for this build
OS	Windows 11	Native Ollama; Docker via WSL2 backend
GPU	NVIDIA Quadro RTX 5000	Turing TU104, 16 GB GDDR6 , compute capability 7.5
CPU	Intel i9-10980XE, 18C/36T	Ample for OCR and CPU-side reranking fallback
RAM	64 GB	Shared with WSL2; cap it explicitly (\$5.5)
Storage	NVMe, 250 GB free	Models ~25 GB; index ~4 KB per chunk

Two properties of the Turing architecture drive decisions throughout:

- **No BF16 and no FP8.** All half-precision work must be FP16.
- **No FlashAttention-2.** FA2 requires SM 8.0 or later. The `llama.cpp` built-in flash-attention kernel does work on Turing and should be enabled; the PyTorch-side FA2 package will not build.

1.4 How to read this

Chapters 2 and 3 are the design rationale and the architecture. Chapters 4 to 10 are the install path, in order. Chapters 11 to 14 cover validation, operations and troubleshooting. Every install step ends in a checkpoint with a stated expected result.

Commands prefixed `PS>` run in PowerShell on the Windows host. Where a command spans lines, the trailing backtick is PowerShell's line continuation.

Chapter 2

Component selection

Each component is listed with what it does, why it was chosen, and the constraint it has to live inside. The binding constraint throughout is 16 GB of VRAM on a Turing GPU: no BF16, no FP8, no FlashAttention-2, and a hard ceiling that all three models plus the Windows desktop must fit under simultaneously.

2.1 The stack

Layer	Component	Licence	Footprint
Generation	Minstral 3 14B Instruct 2512, Q4_K_M GGUF, served by Ollama	Apache 2.0	8.7 GB
Embedding	BAAI BGE-M3, FP16	MIT	1.4 GB
Reranking	BAAI bge-reranker-v2-m3, FP16	Apache 2.0	1.4 GB
Vector store	Qdrant, Docker container	Apache 2.0	~10 MB index
PDF parsing	PyMuPDF, with OCRmyPDF/Tesseract fallback	AGPL-3.0 or commercial	—

2.2 Generation: Minstral 3 14B Instruct 2512

Mistral describes the 14B as the largest model in the Minstral 3 family, with performance comparable to Mistral Small 3.2 24B, deployable locally and fitting in 24 GB of VRAM at FP8 or less when further quantized. It carries an Apache 2.0 licence, a 256k context window, multilingual coverage, strong system-prompt adherence and native JSON output.

Three of those properties are why it suits this application specifically. **Apache 2.0** gives an unambiguous licence for an internal engineering tool. **System-prompt adherence** matters because the entire behaviour of this system is an eight-rule contract in the system prompt; a model that drifts away from it produces uncitable answers. **Reliable structured output** is what lets the citation verifier parse the response deterministically.

At Q4_K_M the weights are 8.7 GB, which leaves room for a 16k context and both retrieval models on the same card.

A naming trap worth recording. Through Mistral Small 3.2, the Small line was a 24B dense model that fits a 16 GB card at 4-bit. Mistral Small 4, released 16 March 2026, is a 119B mixture-of-experts model: 128 experts with 4 active per token, so 6B parameters

active but all 119B resident. Full BF16 weights are roughly 238 GB and 4-bit GGUF lands at 60–70 GB. MoE reduces compute per token, not memory residency, so it fits neither 16 GB of VRAM nor 64 GB of system RAM once Windows, Docker, Qdrant and the retrieval models are present.

For a larger GPU, Mistral Small 3.2 24B at Q4_K_M is the next step up. It does not fit here: 14.3 GB of weights leaves no room for KV cache, compute buffers and the desktop reserve, let alone the retrieval models.

2.3 Embedding: BGE-M3

BGE-M3 supports 100+ languages and performs dense, sparse and ColBERT-style multi-vector retrieval in a single model with an 8,192-token context window. On the Milvus long-context benchmark it degrades only 8% at long input lengths against 56% for nomic-embed-text.

The sparse channel is the reason it was chosen. Standards retrieval is unusually lexical: queries carry exact identifiers such as CS 25.1309, Category 3, IP67, MIL-STD-810H Method 514.8. Dense embeddings are systematically weak at exact-token matching, so a dense-only model needs a separate SPLADE or BM25 pipeline bolted alongside it, with two indexing paths to maintain. BGE-M3 emits both representations from one forward pass, which maps directly onto Qdrant’s named-vectors and sparse-vectors hybrid query.

The 8,192-token window is the second reason. Measured on CS-25, the median chunk body is 2,136 characters — roughly 530 tokens — with a 90th percentile of 2,392. A model capped at 512 tokens would have truncated more than half the corpus, and a truncated clause loses the heading that identifies it, which is precisely what clause-level citation depends on.

One caveat, stated so the choice is made with open eyes: BGE-M3’s English-only MTEB score of 63.2 in the January 2026 snapshot sits below proprietary flagships and well below Qwen3. For a purely English corpus where a separate sparse pipeline is acceptable, Qwen3-Embedding-0.6B scores higher on dense retrieval. The trade made here is a few points of English dense quality for single-model hybrid, multilinguality and long chunks.

2.4 Reranking: bge-reranker-v2-m3

A pointwise cross-encoder of approximately 0.6B parameters with multilingual training objectives. Unlike the embedder, which must encode the query and the corpus independently, a cross-encoder sees the query and one candidate passage together and can therefore model their interaction directly. That is why it is far more accurate and far too slow to run over a whole corpus — hence the two-stage shape in §3.1.

A 2026 study of adaptive re-ranking found it gave better accuracy than the alternatives evaluated, including Qwen3-Reranker-0.6B and Qwen3-Reranker-8B, across all datasets used. The Qwen3 authors report the opposite ordering on their own benchmarks: Qwen3-Reranker-0.6B scores 65.80 on MTEB-R against 57.03 for BGE-reranker-v2-m3, and 66.36 against 58.36 on MMTEB-R. Both are the same size and drop-in at the same interface, so §15.4 includes the harness to decide on your own golden set rather than on either leaderboard.

It is also family-matched to the embedder: both are XLM-RoBERTa-large derivatives at 568M parameters, with the same tokenizer behaviour, the same multilingual coverage and near-identical VRAM footprints. You size once and provision twice.

Its scores also drive the abstention gate, which is explained in §11.2.

2.5 Vector store: Qdrant

Chosen for three properties this application needs. **Named vectors** let one point carry both a dense and a sparse representation, which is the mechanism behind hybrid retrieval. **Reciprocal Rank Fusion** is built into the Query API, so both channels are searched and fused in a single round trip. **Payload indexes** make filtered search fast, which matters for restricting a query to one document or to current editions only.

It runs in Docker with no GPU requirement and stores roughly 4 KB per chunk, so the index is not a resource constraint at this scale.

2.6 Parsing: PyMuPDF

Fast, and it exposes the layout detail the chunker needs: text blocks with their coordinates, font sizes and weights, plus table extraction and page labels. Heading detection depends on typographic prominence, which is only available from a parser that reports span-level font information.

Two things it does not solve on its own. Scanned documents return no text at all, so `trriage.py` checks text coverage and `OCRmyPDF` repairs image-only pages before indexing. And every publisher's house style needs its own heading patterns — chapter 10 covers this as the main effort in adding a new corpus.

PyMuPDF is dual-licensed under AGPL-3.0 and a commercial agreement from Artifex. A single-user workstation tool is normally comfortable under AGPL; the moment the system is reachable over the network by other engineers, AGPL §13 becomes arguable. Get a decision before building on it. If legal declines, `pypdfium2` (BSD) is the substitute, with a weaker layout API that will cost clause-detection accuracy.

2.7 One constraint that is not technical

The standards themselves are copyrighted. ISO, IEC, CEN and most national bodies licence per seat or per site, and many licences prohibit loading the text into a shared retrieval system. Record the clearance in the corpus register (§9.1). An auditor is at least as likely to ask about your right to hold the text as about the accuracy of your citations.

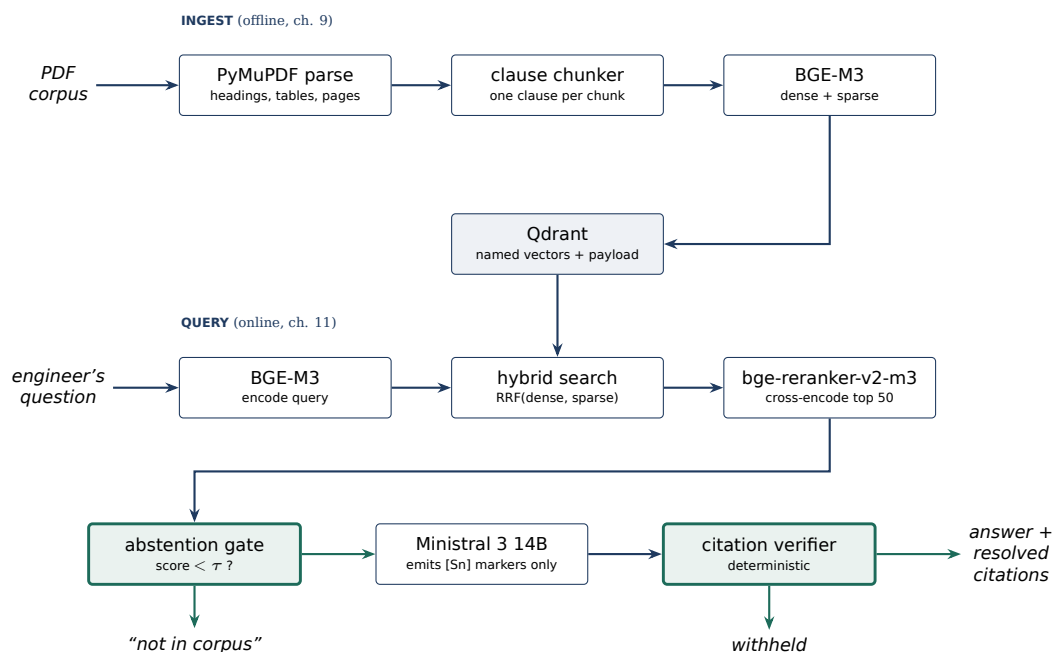
EASA and EUR-Lex material is freely downloadable and reusable, which is one reason CS-25 is a good corpus to build against first.

Chapter 3

Architecture

3.1 The pipeline

Retrieval-augmented generation is a standard pattern, and most of this diagram is the standard pattern. Three components are not, and they are the reason the system can be used as evidence. They are drawn in green.



3.2 Why the LLM appears only once

A common description of this pipeline has the language model embedding the query and handing its answer to the reranker. It does neither.

The **embedding model** encodes the query. **Qdrant** retrieves candidates. The **reranker** scores (query, candidate_passage) pairs — before any generation happens, and it never sees a generated answer. Only then, if the gate allows it, does the LLM generate. A reranker applied after generation would be a fact-checker, which is a different component and is here replaced by the deterministic verifier.

3.3 The three components that make this auditable

The abstention gate is control flow, not a prompt. After reranking, if the top score falls below `TAU_ABSTAIN`, the LLM is never invoked. There is no opportunity to confabulate because no generation occurs. Prompt wording asking a model to say “I don’t know” is a request; this is a branch. That difference is what turns “the system says so when the corpus lacks the answer” from an aspiration into a property of the code.

The model emits markers, never citations. Retrieved passages are presented as [S1], [S2], and rule 7 of the answer contract forbids the model from writing any document number, clause number, page number or date. The application resolves each marker against the Qdrant payload — the same payload the chunker wrote from the parsed PDF — and renders the real citation itself.

The consequence is that **page and clause numbers cannot be hallucinated**, because the model is never in a position to generate one. This closes the most common failure of citation-oriented RAG, in which a model with the correct passage in front of it nevertheless writes a page number two digits off. No amount of prompting fixes that reliably. Removing the opportunity does.

The citation verifier is deterministic and runs after generation. It checks that every marker is in range, that every factual sentence carries one, that every quotation appears verbatim in a cited source body, and that no literal page or clause reference was written. On failure the answer is regenerated once with the problems fed back, then withheld. A withheld answer is a third state, distinct from both an answer and an abstention, and the interface shows it as such.

3.4 What the architecture does not do

Stated plainly, because these are the boundaries an auditor will probe.

It does not read figures. Icing envelopes, graphs and scanned diagrams are images; PyMuPDF extracts text. A question whose answer is a curve returns `not_in_corpus`, which is correct behaviour and not a defect, but it is a real limit.

It does not reason beyond the retrieved text. Rule 2 of the contract forbids inference beyond what a passage states. Questions requiring synthesis across clauses will be answered only where both clauses were retrieved and both are cited.

It does not know what it failed to index. Coverage is measured separately (§9.7), not inferred at query time. A clause the parser missed is indistinguishable, from inside the system, from a clause that does not exist.

Chapter 4

Layout, ports and directories

4.1 Process map

#	Service	Runs as	Port	Purpose
1	Qdrant	Docker container	6333 / 6334	Vector and payload store
2	Ollama	Windows service	11434	Minstral 3 14B generation
3	retrieval-svc	NSSM service	8081	BGE-M3 encode, reranker score
4	rag-api	NSSM service	8080	Orchestration, gate, verifier, web UI
5	Ingestion CLI	On demand	—	Parse, chunk, embed, upsert

Encode/rerank is kept separate from orchestration deliberately. The retrieval service holds about 2.8 GB of GPU-resident weights and takes 20–40 seconds to load. Orchestration holds no state and starts in about a second, so thresholds and prompts can be changed without paying the model load each time.

4.2 Directory layout

```
C:\rag\
├─ corpus\
│  └─ raw\          original PDFs, never modified
│  └─ ocr\          OCR-repaired copies of scanned PDFs
│  └─ registry.yaml  document register: IDs, editions, page offsets
├─ models\          HF cache (HF_HOME points here)
├─ qdrant\
│  └─ docker-compose.yml
│  └─ .env           QDRANT_API_KEY
│  └─ snapshots\     bind-mounted backups, visible from Windows
├─ services\
│  └─ retrieval_svc.py
│  └─ rag_api.py
│  └─ Modelfile
│  └─ prompts\answer_contract.txt
│  └─ static\index.html  web UI
├─ ingest\          parse, chunk, index, triage, verify, probe
└─ eval\            golden.yaml, run_eval.py, xml_coverage.py
```

```
├─ logs\                audit trail and service logs
└─ .venv\
```

Note there is no qdrant\storage\. Vector data lives in a Docker named volume, for reasons given in §6.1.

```
PS> New-Item -ItemType Directory -Force -Path `
    C:\rag\corpus\raw, C:\rag\corpus\ocr, C:\rag\models, `
    C:\rag\qdrant\snapshots, C:\rag\services\prompts, C:\rag\services\static, `
    C:\rag\ingest, C:\rag\eval, C:\rag\logs
```

Then unpack rag-scripts.zip so that ingest\, services\ and eval\ land under C:\rag\. Appendix D lists every file and what it does.

Chapter 5

Host preparation

5.1 NVIDIA driver and CUDA runtime

```
PS> nvidia-smi
```

Checkpoint. The GPU reads *Quadro RTX 5000*, memory 16384MiB, CUDA 12.1 or higher. Note the idle VRAM figure; on a workstation driving a monitor it is typically 0.5–1.0 GB and is the desktop line in the budget of \$7.1.

To recover VRAM, drive the display from integrated graphics if available, or disable hardware acceleration in browsers and Teams. On a 16 GB budget, 400 MB reclaimed from a browser is a real gain.

5.2 Python

Install **Python 3.11**. Tick *Add python.exe to PATH* and *Disable path length limit*.

```
PS> python --version
PS> python -m venv C:\rag\.venv
PS> C:\rag\.venv\Scripts\Activate.ps1
```

If activation is blocked:

```
PS> Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

5.3 Set HF_HOME before anything downloads

Do this **now**, before any model is pulled. `SetEnvironmentVariable` at Machine scope affects only newly launched processes, so the current shell needs it too. Setting it after the first download leaves 4.5 GB of weights in `C:\Users\<you>\.cache\huggingface`, where they are per-user, on the system drive, and outside your backup and antivirus exclusions.

```
PS> [Environment]::SetEnvironmentVariable("HF_HOME", "C:\rag\models", "Machine")
PS> $env:HF_HOME = "C:\rag\models"
```

Checkpoint, in a new shell:

```
PS> python -c "from huggingface_hub import constants; print(constants.HF_HUB_CACHE)"
```

Expect `C:\rag\models\hub`. The hub subdirectory is normal — `HF_HOME` is the parent.

If weights are already in the wrong place, move them with `robocopy` rather than `Move-Item`, because the HF cache uses links from `snapshots\` into `blobs\`:

```
PS> robocopy "$env:USERPROFILE\.cache\huggingface" "C:\rag\models" /E /MOVE
```

5.4 Supporting tools

```
PS> winget install --id Git.Git -e
PS> winget install --id UB-Mannheim.TesseractOCR -e
PS> winget install --id NSSM.NSSM -e
PS> [Environment]::SetEnvironmentVariable("TESSDATA_PREFIX", `
    "C:\Program Files\Tesseract-OCR\tessdata", "Machine")
```

Install the Tesseract language packs the corpus needs: at minimum eng, plus fra and deu for European standards.

5.5 Docker Desktop and the WSL2 memory cap

WSL2 defaults to claiming up to half of system RAM and does not readily give it back. Cap it so it cannot starve the GPU-adjacent Python processes.

Create C:\Users\<you>\.wslconfig:

```
[wsl2]
memory=12GB
processors=8
swap=4GB
localhostForwarding=true
```

12 GB is generous: the HNSW index for the CS-25 corpus is well under 2 GB.

```
PS> wsl --shutdown
PS> wsl -d docker-desktop -e sh -c "free -h"
```

Restart Docker Desktop afterwards.

5.6 Defender exclusions and power plan

```
PS> Add-MpPreference -ExclusionPath "C:\rag"
PS> Add-MpPreference -ExclusionProcess "ollama.exe"
```

Confirm with your IT security policy first; on a managed workstation this may need a sanctioned exception rather than a local command.

Set the power plan to *High performance*. On the balanced plan the CPU down-clocks during the long single-threaded stretches of PDF parsing and costs 20–30% of ingestion wall time.

Chapter 6

Qdrant

6.1 Storage: use a named volume, not a bind mount

On Windows, bind-mounting a host directory into a Linux container routes every file operation through the WSL2 9p translation layer. For Qdrant's segment files that is slow, and it is a recurring source of permission problems. A Docker named volume lives on the WSL2 ext4 disk and avoids both.

```
PS> docker volume create qdrant-storage
PS> docker volume inspect qdrant-storage
```

Snapshots stay a bind mount. They are your backups, and you want them visible in Explorer and covered by ordinary file backup.

6.2 Compose file

Save as C:\rag\qdrant\docker-compose.yml:

```
services:
  qdrant:
    image: qdrant/qdrant:v1.15.1
    container_name: rag-qdrant
    restart: unless-stopped
    ports:
      - "127.0.0.1:6333:6333"
      - "127.0.0.1:6334:6334"
    volumes:
      - qdrant-storage:/qdrant/storage
      - ./snapshots:/qdrant/snapshots
    environment:
      QDRANT__SERVICE__API_KEY: "${QDRANT_API_KEY}"
      QDRANT__SERVICE__ENABLE_TLS: "false"
      QDRANT__STORAGE__SNAPSHOTS_PATH: "/qdrant/snapshots"
      QDRANT__STORAGE__PERFORMANCE__MAX_OPTIMIZATION_THREADS: "4"
      QDRANT__LOG_LEVEL: "INFO"
    healthcheck:
      test: ["CMD-SHELL", "bash -c ':> /dev/tcp/127.0.0.1/6333' || exit 1"]
      interval: 20s
      timeout: 5s
      retries: 5

volumes:
  qdrant-storage:
```

```
external: true
```

Four deliberate choices:

- **volumes: is a top-level key**, a sibling of services:, not nested inside it.
- **external: true** tells Compose the volume already exists. Omit it and Compose silently creates its own volume named qdrant_qdrant-storage and ignores yours.
- **Ports bound to 127.0.0.1**. Without the loopback prefix Docker publishes on 0.0.0.0, and a store holding licensed standards text becomes reachable from the office LAN. This is the commonest misconfiguration in local RAG builds.
- **A pinned image tag**. latest will eventually pull a version with a changed storage format and cost a re-index. Verify the current stable tag on Docker Hub at install time; v1.15.1 is a placeholder.

6.3 Start and verify

```
PS> cd C:\rag\qdrant
PS> $key = -join ((48..57)+(97..122) | Get-Random -Count 40 | % {[char]$_})
PS> Set-Content -Path .env -Value "QDRANT_API_KEY=$key"
PS> docker compose config
PS> docker compose up -d
```

docker compose config validates the YAML before anything starts. Pulling the image first with docker pull qdrant/qdrant:v1.15.1 is optional but separates a download failure from a configuration failure, and fails loudly if the pinned tag does not exist.

Checkpoint.

```
PS> docker compose ps
PS> docker compose port qdrant 6333
PS> docker inspect rag-qdrant --format "{{range .Mounts}}{{.Type}} {{.Name}}{{.Source}} ->
  ↳ {{.Destination}}{{println}}{{end}}"
PS> $env:QDRANT_API_KEY = $key
PS> Invoke-RestMethod http://127.0.0.1:6333/healthz -Headers @{ "api-key" = $key }
```

Expect Up (healthy) after 20–30 seconds, 127.0.0.1:6333 from compose port (not 0.0.0.0), a volume qdrant-storage -> /qdrant/storage line plus a bind line for snapshots, and healthz check passed.

Persistence check. Worth doing once now rather than discovering it later:

```
PS> docker compose down
PS> docker compose up -d
PS> Invoke-RestMethod http://127.0.0.1:6333/collections -Headers @{ "api-key" = $key }
```

Collections must survive the restart. Store the key in your password manager; every script reads it from the QDRANT_API_KEY environment variable.

6.4 Note on PowerShell and JSON

Throughout this guide, requests with a JSON body use Invoke-RestMethod rather than curl.exe. PowerShell's handling of quoting when passing arguments to a native executable differs between Windows PowerShell 5.1 and PowerShell 7, and inline escaped JSON ('{"model\":"x"}') frequently arrives at the server truncated, producing an unexpected EOF parse error that looks like a server fault and is not.

Where a body is unavoidable in curl.exe, use a here-string:

```
PS> $body = @'
{"model":"std-rag","prompt":"Say OK","stream":false}
```

```
'@  
PS> curl.exe -s http://127.0.0.1:11434/api/generate -d $body
```

The closing '@ must begin at column 1 of its own line.

Chapter 7

LLM serving

7.1 VRAM budget

16 GB determines quantization, context length and where the retrieval models live. This build uses **Configuration A**: everything resident on the GPU.

Component	Precision	VRAM
Minstral 3 14B Instruct, weights	Q4_K_M GGUF	8.7 GB
KV cache, 16,384 tokens	q8_0 quantized	1.3 GB
llama.cpp compute buffers	—	0.6 GB
BGE-M3 embedder (568M)	FP16 + activations	1.4 GB
bge-reranker-v2-m3 (568M)	FP16 + activations	1.4 GB
Windows desktop / WDDM reserve	—	0.8 GB
Total		14.2 GB
Headroom		1.8 GB

Configuration B moves the retrieval models to CPU (`RETRIEVAL_DEVICE=cpu`), freeing 2.8 GB for a Q5_K_M quant at the cost of 3–6 seconds per rerank on 18 cores. Use it if answer quality rather than latency is the binding constraint.

Mistral Small 3.2 24B does not fit. Q4_K_M weights alone are about 14.3 GB; adding even a minimal KV cache, compute buffers and the desktop reserve exceeds 16 GB before the retrieval models are considered.

7.2 Install and configure Ollama

```
PS> winget install --id Ollama.Ollama -e
PS> [Environment]::SetEnvironmentVariable("OLLAMA_FLASH_ATTENTION","1","Machine")
PS> [Environment]::SetEnvironmentVariable("OLLAMA_KV_CACHE_TYPE","q8_0","Machine")
PS> [Environment]::SetEnvironmentVariable("OLLAMA_KEEP_ALIVE","-1","Machine")
PS> [Environment]::SetEnvironmentVariable("OLLAMA_MODELS","C:\rag\models\ollama","Machine")
PS> Restart-Service -Name Ollama -Force
```

Flash attention and an 8-bit KV cache together roughly halve KV memory, which is what makes a 16k context viable inside the budget above. This is the `llama.cpp` flash-attention kernel, which does run on Turing — not the PyTorch FlashAttention-2 package, which does not.

`OLLAMA_KEEP_ALIVE=-1` keeps the model resident permanently, avoiding an 8–15 second cold load per query. The budget already accounts for it. If the workstation later runs other GPU work, change it to 30m.

7.3 Pull the model and build the serving profile

```
PS> ollama pull hf.co/unsloth/Ministral-3-14B-Instruct-2512-GGUF:Q4_K_M
PS> ollama list
```

The unsloth mirror is used because the first-party mistralai GGUF repository is gated and needs an authenticated Hugging Face token. To use the first-party weights, accept the terms on the model page, run `huggingface-cli login`, and substitute the repository name.

Ollama's defaults are wrong for this application in three ways: the context is too short to hold six reranked clauses, the temperature is too high for a task where invention is the failure mode, and answers are not reproducible. `C:\rag\services\Modelfile` corrects all three; it ships in the archive.

```
PS> ollama create std-rag -f C:\rag\services\Modelfile
PS> ollama run std-rag "Reply with the single word: ready"
```

Temperature is 0.1 rather than 0.0: greedy decoding on quantized weights can loop on repetitive clause text. seed 42 makes the same question reproduce the same answer, which matters when an auditor asks you to re-run a query from six months ago.

Checkpoint.

```
PS> ollama ps
PS> nvidia-smi --query-gpu=memory.used,memory.total --format=csv
```

`ollama ps` must show **100% GPU**. Expect roughly 10.5 GB allocated. A CPU percentage means layers were offloaded and generation will be an order of magnitude slower.

If Ollama reports an unknown model architecture, the build predates Ministral 3 GGUF support. Update Ollama; failing that, fall back to Qwen3-14B and re-check the budget.

Chapter 8

Retrieval service

8.1 PyTorch for Turing

This must be the CUDA 12.1 build. Turing is sm_75, which the cu121 wheels include.

```
PS> C:\rag\.venv\Scripts\Activate.ps1
PS> pip install --upgrade pip
PS> pip install torch==2.5.1 --index-url https://download.pytorch.org/whl/cu121
PS> python -c "import torch; print(torch.__version__, torch.cuda.is_available(),
↪ torch.cuda.get_device_capability(0))"
```

Checkpoint. Expect (7, 5). False for availability means the driver and the wheel disagree; reinstall the driver before continuing.

8.2 Remaining dependencies

```
PS> pip install -r C:\rag\requirements.txt
```

Every version is pinned. An embedding model is not interchangeable with a later revision of itself: if the weights change, every vector in Qdrant is invalid and the whole corpus must be re-indexed. §16.3 covers the versioning policy.

8.3 Pre-download the weights

Do this deliberately rather than discovering a 4.5 GB download inside the first query.

```
PS> python -c "from FlagEmbedding import BGEM3FlagModel; BGEM3FlagModel('BAAI/bge-m3',
↪ use_fp16=True)"
PS> python -c "from FlagEmbedding import FlagReranker; FlagReranker('BAAI/bge-reranker-v2-m3',
↪ use_fp16=True)"
PS> huggingface-cli scan-cache
```

use_fp16=True is required, not optional: Turing has no BF16 path, and FP32 would double both footprints and break the budget. scan-cache shows the exact commit each model resolved to — record those, and pin revision= in production.

8.4 Start and verify

```
PS> uvicorn retrieval_svc:app --host 127.0.0.1 --port 8081 --app-dir C:\rag\services
```

Wait for retrieval-svc ready; model loading takes 20–40 seconds.

Checkpoint.

```
PS> Invoke-RestMethod http://127.0.0.1:8081/health
PS> $r = Invoke-RestMethod -Uri http://127.0.0.1:8081/rerank -Method Post `
    -ContentType "application/json" -Body @{
        query = "random vibration test duration"
        passages = @(
            "The unit shall be subjected to random vibration for 60 minutes per axis."
            "Packaging shall be recyclable."
        )
    } | ConvertTo-Json
PS> $r.scores
```

/health must report "device":"cuda". The first passage should score above 0.9 and the second near zero. If both sit near 0.5 the model has not loaded correctly, and the abstention gate will not work.

Chapter 9

Ingestion

This is where audit-grade citation is won or lost. Everything downstream can only cite what the chunker recorded.

Ingestion is four scripts run in sequence with a checkpoint after each. Scripts are described in Appendix D; only the commands appear here.

9.1 The corpus register and the page-offset problem

C:\rag\corpus\registry.yaml records what you hold. The worked CS-25 entry:

```
documents:
- doc_id: "CS-25"
  title: "Certification Specifications and Acceptable Means of Compliance
        for Large Aeroplanes"
  publisher: "EASA"
  edition: "Amendment 27"
  date: "2021-11-24"
  file: "raw/CS-25 (Amendment 27).pdf"
  lang: "en"
  page_offset: 0
  allow_generic_decimal: false
  licence_note: "Free download from EASA website, no registration"
  licence_cleared_by: "N/A - public"
```

Which fields do work, and which are record-keeping:

Field	Read by code	Consequence if wrong
doc_id	Yes	Chunk identity, citations, filters. Two documents sharing an ID overwrite each other
edition	Yes	Part of the chunk ID. This is what keeps superseded editions distinct
file	Yes	Ingestion fails outright
page_offset / page_map	Yes	Citations fall back to PDF page numbers, which an auditor will reject
allow_generic_decimal	Yes	See §10.3. false for EASA, true for ECSS/ISO/IEC
skip_tables	Yes	Speed only; skips the slowest parsing stage

Field	Read by code	Consequence if wrong
title, publisher, lang	Into payload	Cosmetic, except lang which is indexed
date, licence_note, licence_cleared_by	No	Documentation and the licence register

page_offset deserves emphasis. Standards PDFs usually carry front matter, so PDF page 14 may be printed page 5. The citation must give the **printed** page — the number an auditor sees on the paper copy.

Determine it once per document: find the first page bearing a printed number, compute `offset = printed - pdf_index` with a 1-based index, then spot-check a page near the end. If the two differ, the document has multiple numbering sections; set `page_offset: null` and supply a `page_map` of `{from_pdf, to_pdf, offset}` ranges.

CS-25 Amendment 27 turns out to need `page_offset: 0` — the printed and PDF numbering coincide. Verify rather than assume; a zero offset that happens to be right is indistinguishable from an offset nobody checked.

Where a PDF carries its own page labels, the parser records them as well. Composite labels such as 1-F-12 cannot be expressed as an integer offset, and the citation falls back to the label.

9.2 Step 1 — confirm the text layer

```
PS> python C:\rag\ingest\trriage.py
```

Checkpoint. One line per PDF. Look for ok, empty pages under 15%, and chars-per-page in the low thousands. Near-zero characters means an image-only PDF; repair it before indexing:

```
PS> ocrmypdf --skip-text --language eng+fra --output-type pdf --optimize 1 `
    "C:\rag\corpus\raw\OLD-STD.pdf" "C:\rag\corpus\ocr\OLD-STD.pdf"
```

`--skip-text` OCRs only the image-only pages. Record in the register that the document is OCR-derived: OCR confuses 0/0 and 1/1, which matters when the token in question is a clause number.

9.3 Step 2 — validate the parser against the publisher

```
PS> cd C:\rag\ingest
PS> python parse.py
PS> python parse.py "C:\rag\corpus\raw\CS-25 (Amendment 27).pdf"
```

The first runs the pattern self-test with no PDF involved — 29 cases covering every supported heading grammar. The second prints block, heading and table counts plus the first and last headings found.

Checkpoint. Headings in the high hundreds to low thousands. Identifiers read CS 25.303, AMC 25.1309, Subpart F, Appendix H, H25.1, with the publisher prefix retained. Levels nest sensibly. Titles are titles, not sentence fragments.

If a whole class of heading is missing, go to §10 before proceeding. Iterating here costs seconds; the same defect found after indexing costs a full re-ingest.

For a document with many tables, add `skip_tables=True` for this check only — `find_tables()` runs at 2–10 pages/s against 30–80 for text, and on a 1,150-page document that is several minutes of silence.

9.4 Step 3 — dry run

```
PS> python index.py C:\rag\corpus\registry.yaml --dry-run
```

Checkpoint, four things:

- **The collision guard stays silent.** It aborts if unique chunk IDs do not equal the chunk count. See §10.4.
- dropped N short/contents entries appears, with low page numbers in the sample.
- no clause under 10%, no page under 2%.
- The sample chunk shows `clause_path` reading Book 1 > Subpart F > CS 25.1309 ... and text opening with that same header before the clause prose.

9.5 Step 4 — ingest

```
PS> python index.py C:\rag\corpus\registry.yaml
```

Expect 10–25 minutes for 1,150 pages. Table detection dominates; embedding roughly 2,400 chunks takes only a couple of minutes. The progress bar appears only during the embed-and-upsert phase, so the first several minutes are silent while parsing runs. That is not a hang.

Finishes with done: N chunks written, collection now holds N points; the two numbers must match.

9.6 Step 5 — verify coverage

```
PS> python verify_ingest.py
```

Exits non-zero and names the failing criterion.

Metric	Target	If it fails
Chunks with no clause number	< 10%	Heading detection failing; §10
Chunks with no printed page	< 2%	page_offset wrong, or a page_map needed
Median body length	800–2,400 chars	Under 400: over-fragmentation. Over 3,000: headings missed
Chunks per page	roughly 2–4	Far below: text extraction failed

Worked result for CS-25 Amendment 27: 2,391 chunks, median body 2,136 characters, 90th percentile 2,392, maximum 4,799, content types 1,357 prose and 1,034 mixed.

Two observations worth carrying forward. The median sits close to `TARGET_CHARS` (2,400) and the maximum is at the `MAX_CHARS` ceiling (4,800), which says CS-25 clauses are large and many are being split on size rather than structure. And a third of chunks are `mixed`, meaning they contain a table — these are among the most valuable content in a qualification corpus and among the worst-served by naive extraction, which is why table detection is worth its cost.

9.7 Step 6 — measure what you missed

`verify_ingest.py` can only see what you produced. It will report healthy figures for a document where the parser silently missed an entire Subpart. An independent statement of what *should* exist is the only way to catch that.

For CS-25 the EASA eRules XML provides one. It is **not ingested**; it is read once, here.

```
PS> python C:\rag\eval\xml_coverage.py "C:\rag\corpus\CS-25 ... xml (machine).xml"
```

Checkpoint. Under 5% missed. Read the sample before reacting:

- A contiguous run of identifiers means one Subpart's heading style defeated detection. A real defect.
- Scattered singles mean the prominence threshold is marginal.
- Short or [Reserved] clauses appear as missed because `MIN_BODY_CHARS` drops them deliberately. Correct behaviour reported as failure.

Worked result: 5.7% missed, accepted. Both sides are reduced to base form before comparison — the XML carries no (g) sub-paragraph suffix while the index does, and comparing raw strings counts every `AMC 25.101(g)` as a phantom gap. Structural headings (Book 1, Subpart F, H25.x, CS 25J901) appear in the extra list because they have no equivalent in the XML identifier set.

9.8 Step 7 — the manual spot-check

Everything above is arithmetic. This is the proof.

Pick three chunks — one early, one mid-document, one in an appendix. For each: open the PDF at the stated **PDF** page, confirm the clause and text are there, then read the printed number in the footer and confirm it matches the citation.

Do this by hand. No automated check substitutes for it, because every automated check shares the assumption being tested. The appendix sample matters most: it is furthest from wherever the offset was measured, so it is where a second numbering section shows up.

Then snapshot before tuning anything:

```
PS> Invoke-RestMethod -Method Post -Headers @{ "api-key" = $env:QDRANT_API_KEY } `
    http://127.0.0.1:6333/collections/standards_v1/snapshots
```

Chapter 10

Adapting the parser to a publisher

Every publisher has a house style, and the parser must be taught each one. This chapter uses CS-25 as the worked case; the same method applies to ECSS, MIL-STD or an internal specification. Expect this to be the main effort when adding a new corpus.

10.1 The diagnostic loop

`probe_heading.py` dumps every block on a page with its font size, bold flag, the filters it tripped and the verdict `_classify()` returned — using your patched parser, so it reflects your configuration rather than an assumption.

```
PS> python probe_heading.py "<pdf>" "Instructions for Continued Airworthiness" --min-page 1000
PS> python probe_heading.py "<pdf>" --page 1313,1314
```

`--min-page` skips the contents section, which otherwise consumes the first dozen matches. `--page` targets a page directly.

Compare a working case against a broken one. If Appendix C indexes and Appendix H does not, dumping both tells you what differs faster than reasoning about it.

10.2 What CS-25 required

Five distinct problems, each found this way:

Alphanumeric numbering. The generic decimal pattern expects 7.2.3. Every CS-25 heading begins with a letter: CS 25.1309, AMC1 25.571, Subpart F, Appendix H. Five new patterns were added.

The publisher prefix must stay inside the identifier. CS 25.1309 and AMC 25.1309 are different clauses — the requirement and its means of compliance. Capturing only the number makes them collide on the same chunk UUID, and one silently overwrites the other.

Small-capped headings. The most instructive failure. CS-25 renders structural headings in small caps: a full-size initial followed by smaller capitals, which PyMuPDF returns as separate spans. Joining spans with a blank produced A PPENDIX H and no pattern could match it. The parser now compares each span's left edge against the previous span's right edge and inserts a space only where the geometry shows a real word gap. This affected every small-capped heading in the document, so whole regions were being absorbed as body text with no error anywhere.

Sub-sections with their own scheme. Appendix H divides into H25.1 through H25.4. These are globally unique — only one H25.1 exists in CS-25 — so they stand as clause identifiers in

their own right at level 3, nesting under Appendix H. Subpart J uses CS 25J901, with no decimal point, which the rule pattern now also accepts.

Sub-paragraph markers must never be headings. (1), (a), (iii) are structure inside a clause. They are rejected unconditionally, in any document.

10.3 The generic decimal trap

This one is worth stating separately because the failure was silent and severe.

AMC clauses carry internal numbered sections: 1 Purpose, 2 Related regulations. The generic decimal pattern, written for ECSS-style numbering where 1 Purpose genuinely is a clause, matched every one of them. Two consequences: phantom clauses with identifiers like 1, 10, 12; and worse, because each false heading triggers a flush, the parent AMC 25.1309 was cut into a dozen fragments — and the clause identifier disappeared entirely, since the AMC heading was immediately followed by 1 Purpose and the flush between them had no body.

The fix is the registry flag `allow_generic_decimal`: `false`, per publisher. Set it for EASA. Leave it `true` for ECSS, ISO and IEC, where decimal numbering is the real scheme.

10.4 The collision guard

`index.py` aborts if unique chunk IDs do not equal the chunk count, because that difference means chunks are overwriting each other and the loss is otherwise silent. Three causes, distinguished by the page numbers in the diagnostic output:

Page pattern	Cause	Remedy
Consecutive — [1174, 1175, 1176]	Running page headers repeating the clause identifier	Absorbed into the open clause rather than opening a new one
Low numbers — [24, 35, 48]	Contents entries	TOC_TAIL_RE plus MIN_BODY_CHARS
Far apart — [22, 918]	Genuine recurrence, e.g. an appendix restating a clause	Occurrence counter in the chunk ID

All three are handled in the shipped parser and chunker. The table is here so a future corpus can be diagnosed the same way.

10.5 Sub-paragraph suffixes

EASA publishes AMCs targeted at specific sub-paragraphs: AMC 25.101(g), AMC 25.101(h)(3). The suffix is real citation precision and is kept in `clause_no`. A parallel `clause_base` field carries the stripped form for grouping, filtering and coverage comparison. Evaluation matches on base form, so gold CS 25.803 matches a returned CS 25.803(c); the publisher prefix remains significant, so AMC 25.803 does not.

Chapter 11

Query pipeline

11.1 Hybrid retrieval

Qdrant's Query API runs both channels as prefetches and fuses them with Reciprocal Rank Fusion in one round trip. RRF rather than a weighted score sum: dense cosine similarity and sparse lexical scores are not on a common scale, so any fixed weighting is wrong for some queries. RRF combines ranks and sidesteps calibration.

Retrieve 50, rerank, keep 6. The wide-then-narrow shape is the point of a two-stage pipeline: the bi-encoder is cheap and recall-oriented, the cross-encoder expensive and precision-oriented. Retrieving only 10 wastes the reranker, because the passage you needed was at rank 23.

A diversity cap of 3 chunks per clause and 4 per document stops one long clause consuming every slot.

11.2 The abstention thresholds, explained

These two numbers decide whether the system answers. They are worth understanding rather than inheriting.

11.2.1 What a reranker score actually is

The cross-encoder reads the query and one candidate passage together and emits a single logit. The service calls `compute_score(..., normalize=True)`, which applies a sigmoid, so what reaches the pipeline is a number between 0 and 1.

The normalisation is not cosmetic. A raw logit is not comparable across queries: a logit of 2.0 might be the best match for one question and a poor one for another, because the scale shifts with the query. The sigmoid maps everything onto a fixed range where a fixed threshold is meaningful. **If `normalize=True` were dropped, the thresholds below would stop working** and the symptom would be either every question abstaining or none of them.

Interpret the score as the model's confidence that this passage answers this query. In practice on a standards corpus it falls into fairly distinct bands:

Score	What it usually means
0.90 - 1.00	The passage directly answers the question
0.60 - 0.90	Relevant and probably contains the answer, or part of it
0.30 - 0.60	Same topic, does not answer the question
0.05 - 0.30	Same vocabulary, different subject
below 0.05	Unrelated

The 0.30–0.60 band is where the difficulty lives. That is exactly where an *adjacent negative* lands: a question about cabin crew complement retrieving CS 25.807 on emergency exits. Same aircraft, same vocabulary, wrong instrument, no answer. A system that answers from that band fabricates confidently.

11.2.2 TAU_ABSTAIN = 0.35 — whether to answer at all

After reranking, the candidates are sorted by score. If the **top** score is below TAU_ABSTAIN, the pipeline returns `not_in_corpus` and **the LLM is never called**.

0.35 sits just above the top of the “same topic, no answer” band. Raising it makes the system more cautious: fewer fabrications, more false abstentions. Lowering it makes the system more willing to answer from weak evidence.

The two failure modes are not symmetric, and this is the reason the default sits where it does:

- A **false abstention** costs an engineer two minutes. They rephrase, or they open the PDF.
- A **confident wrong citation** can propagate into a design review, a compliance matrix or a qualification report, and it will be discovered by someone else.

So the threshold is set to be slightly too cautious on purpose.

11.2.3 TAU_INCLUDE = 0.15 — which passages the model sees

Once the gate has decided to answer, TAU_INCLUDE filters the remaining candidates. Anything scoring below it is discarded rather than shown to the model, up to the KEEP limit of 6.

This is a different job from TAU_ABSTAIN and it is easy to conflate the two. The gate asks “is there an answer here at all?”. TAU_INCLUDE asks “is this particular passage worth the model’s attention?”.

It exists because padding the context with weakly relevant clauses measurably increases citation errors. When the correct passage is thin, a plausible-looking irrelevant passage is what a model reaches for. Six mediocre passages produce worse answers than two good ones, so the filter is deliberately more aggressive than it needs to be for context-length reasons.

0.15 is low because by this point the gate has already confirmed a strong top hit. The passages between 0.15 and 0.35 are supporting material: the AMC beside the CS clause, the appendix the clause refers to. Those are often exactly what a multi-source question needs.

11.2.4 A worked example

Question: “Within what time must the maximum seating capacity be evacuated?” Reranked candidates:

Rank	Clause	Score	Fate
1	CS 25.803(c)	0.97	Top score above 0.35, so the gate opens. Shown as [S1]
2	AMC 25.803	0.81	Above 0.15, shown as [S2]
3	CS 25.807	0.34	Above 0.15, shown as [S3]
4	Appendix J	0.22	Above 0.15, shown as [S4]
5	CS 25.812	0.11	Below 0.15, discarded
6–50	various	< 0.11	Discarded

Now the adjacent negative: “How many cabin crew are required for 200 seats?”

Rank	Clause	Score	Fate
1	CS 25.807	0.31	Below 0.35. Gate closes, LLM never called
2	CS 25.803	0.28	Never considered

The answer is `not_in_corpus` with `best_score`: 0.31 reported. Note how close that is to the threshold — 0.31 against 0.35. That margin is thin, and it is exactly what §15.4 tells you to watch.

11.2.5 How to calibrate

Do not tune these by intuition. Use the golden set:

1. Run `run_eval.py` and note every fabrication (a negative that got answered) and every false abstention (a positive that got refused).
2. `run_eval.py` reports the **highest score among correctly refused negatives**. If that number sits within about 0.05 of `TAU_ABSTAIN`, the margin is thin: raise the threshold even though the evaluation passed.
3. For each fabrication, read the `best_score` in the log line. Raise `TAU_ABSTAIN` above it, in 0.05 steps.
4. Re-run. Confirm false abstentions have not crossed the 15% gate.
5. Repeat. The two metrics move in opposition; the golden set is what tells you where to stand.

Both are environment variables on the `rag-api` service, so changing them needs no re-index and no restart of the retrieval service:

```
PS> nssm set RagApi AppEnvironmentExtra "QDRANT_API_KEY=$key TAU_ABSTAIN=0.40"
PS> nssm restart RagApi
PS> Invoke-RestMethod http://127.0.0.1:8080/health
```

`/health` echoes the thresholds in force, which is how you confirm a change took effect.

11.2.6 The other two constants

`CANDIDATES = 50` is how many passages hybrid search retrieves before reranking. The wide-then-narrow shape is the point of a two-stage pipeline: the bi-encoder is cheap and recall-oriented, the cross-encoder expensive and precision-oriented. Retrieving only 10 wastes the reranker, because the passage you needed was at rank 23. Raise it to 80 if the golden set shows the gold clause is being retrieved but not reranked into the top 6.

`KEEP = 6` is how many survive into the context. Six chunks at CS-25’s median of 2,136 characters is roughly 5,000 tokens including the contract, comfortably inside the 16k window. A diversity cap of 3 chunks per clause and 4 per document stops one long clause consuming every slot.

11.3 Citation verification

Deterministic, after generation, before the answer reaches the engineer. Prompting alone never achieves audit-grade citation, so four checks run on the completed text:

1. **Every marker is in range.** A [57] when only six sources were supplied is a fabricated reference.
2. **Every factual sentence carries a marker.** Sentences under 25 characters and list headers are exempt; anything substantive is not.
3. **Every quotation appears verbatim in a cited source body.** Whitespace-normalised, and checked against `body` rather than the embedded text — the synthetic clause-path header would otherwise be quotable, letting the model “cite” a string that appears nowhere in the standard.

4. **No literal page or clause reference was written**, per rule 7 of the contract.

On failure the answer is regenerated once with the problem list appended to the prompt. If it fails again the answer is **withheld**, and the retrieved clauses are listed for manual review instead. Withheld is a distinct state from abstention: the difference between “no passage answers this” and “a passage does, but the answer could not be validated”, and the interface shows them differently.

Quotation failures are treated as the most serious defect. A fabricated quotation indicates the model is not tracking the source text at all.

11.4 Audit trail

Every query appends one JSON line to `C:\rag\logs\queries.jsonl`: timestamp, question, status, answer, citations with scores, latency, model, collection and the threshold in force. This is the record that answers “which clause did the tool cite on this date”. Include it in your normal retention policy.

Chapter 12

The answer contract

The system prompt at `C:\rag\services\prompts\answer_contract.txt` is where this system's behaviour is defined. It is short, and every line in it exists because of a specific failure mode.

12.1 The eight rules

- 1. Every factual sentence must end with one or more markers of the form [S1] or [S2][S4].** A factual sentence without a marker is not permitted. This is what the verifier checks first, and it makes unsourced assertion detectable rather than merely discouraged.
- 2. Assert nothing not supported by the text of a cited source.** No general engineering knowledge, no inference beyond what the passage states. The model knows a great deal about aircraft certification from its training data, and all of it is unciteable. This rule makes that knowledge off-limits.
- 3. If the sources do not answer the question, reply with exactly NOT_IN_CORPUS: <one sentence naming what would be needed>.** No partial answer alongside it. See §12.3.
- 4. If sources conflict, or apply to different editions or categories, say so explicitly and cite each side.** Standards corpora contain genuine contradictions, particularly across editions. Silently picking one is the worst available behaviour; surfacing the conflict is the right one, and it is often the most valuable answer the system gives.
- 5. Quotations must be verbatim and in double quotation marks. Quote sparingly; prefer precise paraphrase with a marker.** The verifier checks every quotation against the source text, so a fabricated quotation is caught. Sparing use is also a licence control: a tool that reproduces long verbatim passages starts to substitute for holding the standard.
- 6. Preserve normative force exactly.** *shall* is a requirement, *should* a recommendation, *may* permission, *will* a statement of fact. Never substitute one for another, never soften or strengthen. This is the rule most specific to standards work. An answer that reports a *shall* as a *should* is wrong in the way that matters most, and it reads perfectly fluently.
- 7. Never write a document number, clause number, page number or date of your own.** The markers carry them.
- 8. Answer in the language of the question.**

12.2 Why rule 7 is the most important line in the system

The model is shown passages labelled [S1], [S2] and is forbidden from writing any identifier itself. The application resolves each marker against the Qdrant payload — the same payload the chunker wrote from the parsed PDF — and renders the citation.

The consequence is that **page and clause numbers cannot be hallucinated, because the model is never in a position to generate one.**

This closes the most common failure of citation-oriented RAG. A model with the correct passage in front of it will nevertheless, at some rate, write a page number two digits off or attribute a requirement to the neighbouring clause. The output is fluent, the citation is specific, and it is wrong — which is worse than a vague answer, because a specific citation invites trust and gets copied into documents.

No amount of prompting fixes that reliably. Removing the opportunity does. If you edit the contract, keep rule 7 intact.

12.3 Why “not in corpus” matters more than any answer

It is tempting to treat abstention as the degraded case — what the system does when it fails. For this application it is the opposite: **the abstention behaviour is what makes every other answer worth reading.**

Consider what the tool is for. An engineer asks whether a requirement exists and what it says. There are two possible truths: the corpus contains the answer, or it does not. A system that answers the first case well and fabricates in the second is not 90% useful — it is unusable, because the engineer cannot tell which case they are in. Every answer must then be verified against the source, and if you are going to open the PDF anyway, the tool has saved nothing.

A system that reliably says “not in corpus” inverts that. An answer means *there is a passage, and here it is*. A refusal means *look elsewhere*. Both are actionable. The engineer only opens the PDF to confirm, not to check whether the answer was invented.

This is why abstention is enforced in three independent places rather than requested once:

1. **Control flow.** If the top reranked score is below `TAU_ABSTAIN`, the LLM is never invoked (§11.2). No generation, no opportunity to confabulate. This is a branch in the code, not a request to a model.
2. **The contract.** Rule 3 gives the model an explicit, exact output for the case where passages were retrieved but none answers the question. The gate cannot catch this: passages can score highly for relevance and still not contain the answer.
3. **The verifier.** If the model answers anyway and cannot support its sentences with markers, or quotes text that is not in a source, the answer is withheld (§11.3).

Layer 1 handles “nothing relevant was found”. Layer 2 handles “relevant passages exist but none answers this”. Layer 3 handles “the model answered but cannot back it up”. Three different failures, three different mechanisms, and only the second one is a prompt.

The interface reinforces it. Abstention is rendered in calm slate blue, never red, because it is correct behaviour and styling it as an error teaches engineers to read correct behaviour as a bug (§13.2).

The acceptance criteria follow from the same reasoning. Fabrications on out-of-corpus questions are the one metric in §15.3 with **zero** tolerance, while a 15% false-abstention rate is acceptable. One confident wrong citation invalidates the tool for audit purposes, because an auditor who finds one will reasonably assume there are others.

12.4 Editing the contract

Changing the contract requires no re-index and no service restart beyond `rag-api`, which makes it tempting to iterate on. Two cautions.

Keep rules 3, 6 and 7 verbatim. Rule 3 is the abstention output the pipeline parses; rule 6 is the normative-force guarantee; rule 7 is what makes citations structurally sound. The others can be adjusted for tone and format.

Re-run the golden set after any edit. The contract is the largest single influence on both citation accuracy and abstention rate, and a wording change that reads as harmless can move both. Prompt edits are cheap to make and cheap to measure; do the second as well as the first.

Chapter 13

The web UI

The interface is a single self-contained HTML file served by rag-api at `http://127.0.0.1:8080`.

It is served from the API's own origin deliberately. A page opened from the filesystem cannot POST to `127.0.0.1:8080` — the browser blocks it as cross-origin — and the CORS middleware that would permit it would also open the API to any page the user visits.

```
PS> Invoke-RestMethod http://127.0.0.1:8080/health
```

Then open `http://127.0.0.1:8080` in a browser.

13.1 What it shows

A question box, a verdict band, the answer, and a sources table.

The verdict band is the loudest element on the page, because the characteristic property of this system is that it reliably declines. The three states are visually distinct: answered-and-verified in green, withheld in amber, and not-in-corpus in slate blue. Abstention is deliberately not red. It is the system working correctly, and styling it as an error teaches engineers to read correct behaviour as a bug.

[S1] markers in the answer are clickable and scroll to the matching source row. Clause identifiers are set in a monospaced face so CS 25.803(c) and AMC 25.101(h)(3) align in the column and 0/0 remain distinguishable — which matters for OCR-derived text.

A copy button produces question, answer and citations as plain text. A print stylesheet drops the form and chrome and adds the question and a UTC timestamp, producing a record that can be filed against a compliance matrix.

Model output is HTML-escaped before markers are linkified, never injected raw.

13.2 Configuration

RAG_STATIC overrides the directory, default `C:\rag\services\static`. If `index.html` is absent the route returns a 404 naming the path it looked for; the API itself continues to work.

Chapter 14

Running as Windows services

Both Python services run under NSSM so the system survives a reboot and does not depend on an open console window.

Do this **after** the golden set passes its gates. While tuning thresholds and the answer contract, rag-api is better in a window where a traceback appears in front of you; it starts in a second and holds no state. retrieval-svc benefits from being a service immediately, since it loads 2.8 GB of weights.

14.1 The environment problem

rag_api.py reads QDRANT_API_KEY at import time, and a service does not inherit a variable set in a PowerShell window. Pass it to the service directly rather than setting it machine-wide, so the key is not readable by every process on the box.

14.2 retrieval-svc

```
PS> nssm install RagRetrieval C:\rag\.venv\Scripts\python.exe
PS> nssm set RagRetrieval AppParameters "-m uvicorn retrieval_svc:app --host 127.0.0.1 --port
↪ 8081"
PS> nssm set RagRetrieval AppDirectory C:\rag\services
PS> nssm set RagRetrieval AppStdout C:\rag\logs\retrieval.log
PS> nssm set RagRetrieval AppStderr C:\rag\logs\retrieval.err.log
PS> nssm set RagRetrieval AppRotateFiles 1
PS> nssm set RagRetrieval AppRotateBytes 10485760
PS> nssm set RagRetrieval Start SERVICE_AUTO_START
PS> nssm start RagRetrieval
```

14.3 rag-api

```
PS> $key = ((Get-Content C:\rag\qdrant\.env) -replace 'QDRANT_API_KEY=', '')
PS> nssm install RagApi C:\rag\.venv\Scripts\python.exe
PS> nssm set RagApi AppParameters "-m uvicorn rag_api:app --host 127.0.0.1 --port 8080"
PS> nssm set RagApi AppDirectory C:\rag\services
PS> nssm set RagApi AppEnvironmentExtra "QDRANT_API_KEY=$key"
PS> nssm set RagApi AppStdout C:\rag\logs\rag_api.log
PS> nssm set RagApi AppStderr C:\rag\logs\rag_api.err.log
PS> nssm set RagApi AppRotateFiles 1
PS> nssm set RagApi AppRotateBytes 10485760
PS> nssm set RagApi Start SERVICE_AUTO_START
```

```
PS> nssm set RagApi DependOnService RagRetrieval
PS> nssm set RagApi AppThrottle 30000
PS> nssm start RagApi
```

AppEnvironmentExtra takes NAME=value pairs separated by spaces; add TAU_ABSTAIN=0.40 or RAG_MODEL=std-rag-alt the same way to pin settings per service.

DependOnService orders startup, but note it waits only for the *service* to start, not for BGE-M3 to finish loading. On a cold boot rag-api may come up while the encoder is still initialising and the first query will fail. AppThrottle 30000 sets a 30-second restart delay, which covers it.

Checkpoint.

```
PS> Get-Service RagRetrieval, RagApi | Format-Table Name, Status, StartType
PS> Invoke-RestMethod http://127.0.0.1:8081/health
PS> Invoke-RestMethod http://127.0.0.1:8080/health
```

If /health fails, read C:\rag\logs\rag_api.err.log. A missing QDRANT_API_KEY and a missing answer contract both produce an explicit message naming what was not found.

After any change to an environment variable, nssm restart RagApi — services read the environment at start.

14.4 Reboot test

```
PS> Restart-Computer
```

After the machine comes back, without logging into anything or opening a console:

```
PS> docker ps
PS> Get-Service RagRetrieval, RagApi
PS> Invoke-RestMethod http://127.0.0.1:8080/health
```

All three must be healthy. Docker Desktop needs *Start Docker Desktop when you log in* enabled; the compose file's restart: unless-stopped handles the container itself.

Chapter 15

Validation

An installed stack is not a qualified one. This chapter is the difference between “it returns answers” and “we can defend its answers”.

15.1 Build a golden set

Forty questions is a workable minimum. Composition matters more than count:

- **~55% in-corporus, answerable.** Spread across documents, clause depths, and both prose and table content.
- **~20% adjacent negatives.** The topic *is* in the corpus but the specific answer is not. These are the only questions that meaningfully test abstention. A system that abstains on “espresso temperature” and confabulates on a neighbouring standard’s requirement has not solved anything.
- **~15% multi-source.** Answerable only by combining two clauses, or where two editions conflict.
- **~10% far negatives.** Obviously out of scope; a cheap regression tripwire.

CS-25 supplies unusually good adjacent negatives, because neighbouring EASA instruments share its vocabulary almost entirely: cabin crew complement (an operational rule under Regulation 965/2012), engine endurance testing (CS-E), flyover noise limits (CS-36), maintenance check intervals (Part-M). Every one returns confident-looking CS-25 passages from a dense retriever and must still be refused.

The CS + AMC structure also supplies natural multi-source questions: “what does CS 25.1309 require, and how do I show compliance?” needs the requirement from Book 1 and the means of compliance from Book 2, and is exactly what an engineer would type.

Write the questions from clauses you have read. Sample the clause list your ingest actually produced, open each in the PDF, and write a question it answers. A golden set with wrong references reports failures that are defects in the test, and days get spent tuning a pipeline that was working.

15.2 Run it

```
PS> python C:\rag\eval\run_eval.py C:\rag\eval\golden.yaml
PS> python C:\rag\eval\run_eval.py C:\rag\eval\golden.yaml --answers
PS> python C:\rag\eval\run_eval.py C:\rag\eval\golden.yaml --only q003 --answers
PS> python C:\rag\eval\run_eval.py --self-test
```

--answers prints every answer in full with its sources and a per-question verdict, for manual review. --self-test checks the clause-matching rules without needing any service running.

Each question reports STATUS OK or STATUS KO with a reason, and the reason is the diagnosis: gold not cited with a near-miss clause is a reranking problem; expected abstention is a threshold problem; false abstention prints the best score so you can see how far under TAU_ABSTAIN it landed.

15.3 Acceptance thresholds

Metric	Minimum to deploy	Target
Gold clause cited (in-corpus)	80%	> 90%
Correct abstention (out-of-corpus)	95%	100%
Fabricated answers on negatives	0	0
False abstentions (in-corpus)	< 15%	< 8%
Latency p95	< 45 s	< 25 s

The fabrication row has no tolerance. One confident answer to a question the corpus cannot support invalidates the tool for audit purposes, because an auditor who finds one will reasonably assume there are others.

Citation precision is reported but is not a gate. It divides gold hits by every source the model cited, so a question with one gold clause answered from three sources scores 33% however correct it is. *Gold clause cited* is the metric that means something. *Gold cited at rank* is the useful diagnostic: median rank 1 means the reranker is placing the right clause on top; a worst case of 5 or 6 means it is working but marginally.

15.4 Tuning order

Change one thing at a time, in this order. Working backwards from the generator wastes days on problems that are in the index.

1. **Gold clause not among the 50 candidates** — retrieval. Check the chunker first via `verify_ingest.py` and `xml_coverage.py`. Confirm the sparse channel is populated: a point with an empty `lexical` vector means the encode path dropped it.
2. **In candidates but not the top 6** — reranking. Raise CANDIDATES to 80, or A/B Qwen3-Reranker-0.6B against the current model.
3. **In the top 6 but not cited** — generation. Check the context is not being truncated; six chunks at 2,400 characters plus the contract is about 5,000 tokens, comfortably inside 16k, but a table-heavy chunk can be much larger.
4. **Fabrication on negatives** — raise TAU_ABSTAIN in 0.05 steps. Then confirm rule 3 of the contract survived any prompt edits.
5. **Too many false abstentions** — lower TAU_ABSTAIN, but only after confirming the negatives still pass. These two move in opposition; the golden set is what tells you where to stand.

Also watch the highest rerank score among *correctly refused* negatives, which `run_eval.py` reports. If it sits close to the threshold, the margin is thin and TAU_ABSTAIN should rise even though the evaluation passed.

Chapter 16

Operations

16.1 Backup

Snapshots are consistent and cheap. Weekly, and always before a re-index.

```
PS> Invoke-RestMethod -Method Post -Headers @{ "api-key" = $env:QDRANT_API_KEY } `
    http://127.0.0.1:6333/collections/standards_v1/snapshots
PS> Get-ChildItem C:\rag\qdrant\snapshots
```

Restore:

```
PS> Invoke-RestMethod -Method Put -Headers @{ "api-key" = $env:QDRANT_API_KEY } `
    -ContentType "application/json" `
    -Uri "http://127.0.0.1:6333/collections/standards_v1/snapshots/recover" `
    -Body (@{ location = "file:///qdrant/snapshots/<snapshot-name>" } | ConvertTo-Json)
```

Because storage is now a named volume, the raw data is no longer copyable with Explorer. To archive the volume itself:

```
PS> New-Item -ItemType Directory -Force -Path C:\rag\qdrant\backup
PS> docker run --rm -v qdrant-storage:/data -v C:\rag\qdrant\backup:/backup `
    alpine tar czf /backup/qdrant-storage.tar.gz -C /data .
```

Prefer snapshots. A tar of a live volume is a copy of files mid-write; a snapshot is consistent by design.

Back up registry.yaml, golden.yaml and logs\ with ordinary file backup. The register is small and irreplaceable: it holds the page offsets determined by hand and the licence clearances.

16.2 Adding or superseding a standard

Adding: place the PDF, run triage, add a register entry with its page offset and publisher flags, re-run index.py. Deterministic chunk IDs make re-running over the whole register idempotent.

Superseding needs a decision. When Edition C replaces Edition B, **do not delete Edition B**. Hardware qualified against B remains qualified against B, and an engineer investigating a 2021 design needs the 2021 text.

1. Ingest the new edition alongside; the edition field keeps the points distinct.
2. Add status: superseded and superseded_by to the old register entry and re-index it so the payload carries the flag.
3. Default the query filter to current editions; surface superseded hits only on request, or when no current-edition passage clears the gate.
4. When both editions clear the gate and disagree, contract rule 4 requires the model to cite both. That behaviour is desirable; do not suppress it.

16.3 Versioning and re-index policy

The collection name carries a version suffix for a reason. A vector is meaningful only relative to the function that produced it.

Change	Requires full re-index?
New document added	No
Registry <code>page_offset</code> / <code>page_map</code> changed	Yes, but that document only (--only)
Publisher flags (<code>allow_generic_decimal</code> , <code>skip_tables</code>) changed	Yes, that document
Parser version changed	Yes
Chunker parameters (<code>TARGET_CHARS</code> , <code>MAX_CHARS</code> , <code>NAMESPACE</code>) changed	Yes
Embedding model, revision or <code>EMBED_MAX_LEN</code> changed	Yes, into a new collection
Reranker changed	No
LLM changed	No
Prompt or thresholds changed	No

For an embedding change, re-index into `standards_v2`, run the golden set against both, and switch `RAG_COLLECTION` only when v2 wins. Keep v1 until it has.

Pin revision= on the Hugging Face model loads in production. A repository can be updated in place, and a silent weight change under a stable name is the failure you cannot debug from the symptoms.

16.4 Health checks

```
PS> Invoke-RestMethod http://127.0.0.1:6333/healthz -Headers @{"api-key" = $env:QDRANT_API_KEY }
PS> Invoke-RestMethod http://127.0.0.1:8081/health
PS> Invoke-RestMethod http://127.0.0.1:8080/health
PS> ollama ps
PS> nvidia-smi --query-gpu=memory.used,memory.total,utilization.gpu --format=csv
```

`ollama ps` is the one to watch. If it starts showing a CPU percentage where it previously showed 100% GPU, something else has taken VRAM — usually a browser or a second process holding a CUDA context — and generation speed has dropped by an order of magnitude.

Chapter 17

Troubleshooting

Symptom	Likely cause	Action
unexpected EOF from Ollama or an 8081/8080 endpoint torch.cuda.is_available() is False Models downloaded to C:\Users\...\cache	PowerShell mangled inline escaped JSON CPU-only wheel, or driver mismatch HF_HOME set after the first download, or not in the current shell	Use Invoke-RestMethod, or a here-string (§6.4) Reinstall from the cu121 index URL §5.3; move with robocopy /MOVE
CUDA out of memory when reranking ollama ps shows a CPU split	LLM resident plus too large a reranker batch VRAM taken by another process	Lower RERANK_BATCH to 4, or set RETRIEVAL_DEVICE=cpu Close GPU-accelerated browsers; lower num_ctx; stay on Q4_K_M
Ollama reports unknown architecture	Build predates Ministral 3 GGUF support	Update Ollama; else fall back and re-check the budget
Parsing appears to hang for minutes Zero text extracted	find_tables() at 2–10 pages/s Image-only PDF	Expected. Use skip_tables for heading checks trriage.py, then ocrmypdf --skip-text probe_heading.py; §10
Headings found but classified body Phantom numeric clauses (1, 10, 12) Chunk ID collision on ingest	Publisher pattern missing, or small caps Generic decimal pattern firing on AMC sub-sections Running headers, contents entries, or genuine recurrence	allow_generic_decimal: false §10.4; diagnose by the page pattern
Indexed identifier count exceeds the reference Page numbers off by a constant Page numbers right early, wrong later Exact clause query returns nothing	Comparing suffixed against base form Wrong page_offset Multiple numbering sections Sparse vector empty	Both sides reduce to clause_base Recompute; re-index with --only Replace page_offset with a page_map Check a point in the dashboard has a lexical vector
Every answer is not_in_corpus	Reranker scores uncalibrated	Confirm normalize=True; test /rerank directly
Model writes literal page numbers UI loads but requests fail	Contract rule 7 weakened Opened from the filesystem, not the API	Restore it; the verifier is already flagging these Open http://127.0.0.1:8080

Symptom	Likely cause	Action
Service starts, first query fails	rag-api up before models loaded	AppThrottle; retry
Qdrant unreachable after reboot	Docker Desktop not set to auto-start	Enable it; compose already has restart: unless-stopped
WSL2 consuming 30+ GB	.wslconfig absent or not applied	\$5.5, then <code>wsl --shutdown</code>
Compose creates qdrant_qdrant-storage	external: true missing	\$6.2

Chapter 18

Appendix A — Version pin table

Record the exact versions installed. This table plus `registry.yaml` plus a Qdrant snapshot is enough to reconstruct the system.

Component	Version	Licence	Notes
Ministral 3 14B	Q4_K_M GGUF	Apache 2.0	Record repo and revision
Instruct 2512			
Ollama	(record)	MIT	Flash attn on, KV cache q8_0, keep-alive -1
BGE-M3	BAAI/bge-m3	MIT	Pin revision= in production
bge-reranker-v2-m3	BAAI/bge-reranker-v2-m3	Apache 2.0	Pin revision=
Qdrant	v1.15.1 (verify)	Apache 2.0	Never :latest
PyMuPDF	1.24.14	AGPL-3.0 or commercial	\$2.6, Appendix C
PyTorch	2.5.1+cu121	BSD-3	cu121 required for sm_75
FlagEmbedding	1.3.4	MIT	
qdrant-client	1.12.1	Apache 2.0	Query API needs ≥ 1.10
OCRmyPDF	16.6.2	MPL-2.0	Wraps Tesseract
Tesseract	(record)	Apache 2.0	Install corpus languages
NSSM	(record)	Public domain	
Python	3.11.x	PSF	

Chapter 19

Appendix B — Performance on this hardware

Indicative figures for the Quadro RTX 5000 (Turing, 448 GB/s) under Configuration A.

Query latency:

Stage	Config A (GPU)	Config B (CPU retrieval)
Encode query	30-60 ms	150-300 ms
Qdrant hybrid search, 50 candidates	20-80 ms	same
Rerank 50 passages	0.8-1.5 s	4-7 s
Prefill ~5,000 prompt tokens	5-9 s	same
Decode ~250 answer tokens	8-12 s	same
Verify	< 10 ms	same
Total	15-23 s	19-29 s

Decode on a 14B Q4_K_M model is memory-bandwidth-bound at roughly 20-30 tokens/s. If the wall-clock figure is unacceptable, reduce CANDIDATES to 30 and KEEP to 4 before reaching for a smaller model: retrieval stages are cheaper to cut than generation quality is to lose.

Ingestion, measured on CS-25 Amendment 27 (1,150 pages, 2,391 chunks):

Stage	Rate
PyMuPDF text extraction	30-80 pages/s
find_tables()	2-10 pages/s — dominates total time
OCR, image-only pages	1-3 pages/s
BGE-M3 embedding, FP16, batch 8	25-60 chunks/s
Qdrant upsert and index build	2,000-8,000 points/s

Storage: roughly 4 KB per chunk. The CS-25 collection occupies about 10 MB plus HNSW overhead. The 25 GB of model weights dominate; storage is not a constraint at this scale.

Chapter 20

Appendix C — Licence and IP register

Complete before go-live and keep with the system documentation.

Item	Question to answer	Owner
PyMuPDF AGPL-3.0	Is the tool network-accessible to others? Does §13 apply? Is an Artifex commercial licence needed?	Legal
Standards text	Does each licence permit loading full text into a searchable index? Per seat or per site?	Legal / Standards librarian
Redistribution	Does the system return enough verbatim text to substitute for holding a licence?	Legal
Model licences	Ministral 3 (Apache 2.0), BGE-M3 (MIT), bge-reranker-v2-m3 (Apache 2.0) — all permit commercial use	Engineering
Query logs	Do logged questions contain export-controlled or programme-sensitive information? Retention period?	Security
Export control	Is any ingested standard subject to ITAR/EAR or national restriction? Does workstation access control match?	Export compliance

The verbatim-quotation question is the subtle one. Contract rule 5 instructs the model to quote sparingly and the verifier enforces that quotations are real. Consider capping quotation length in the verifier, around 40 words, so the tool cannot be used to reconstruct a standard clause by clause. That cap is a licence control, not a quality one.

Chapter 21

Appendix D — Script reference

All scripts ship in `rag-scripts.zip`. Paths are relative to `C:\rag\`.

Script	Purpose	Key flags	Checkpoint
<code>ingest/triage.py</code>	Text-coverage report per PDF; flags scanned documents	path argument	\$9.2
<code>ingest/parse.py</code>	PyMuPDF parsing into typed blocks; heading grammars per publisher	no args = 29-case self-test; PDF path = summary	\$9.3
<code>ingest/chunk.py</code>	Clause-aware chunking with citation metadata	no args = self-test	\$9.3
<code>ingest/index.py</code>	Parse → chunk → embed → upsert; collision guard	<code>--dry-run</code> , <code>--only DOC_ID</code>	\$9.4, \$9.5
<code>ingest/verify_ingest.py</code>	Post-ingest coverage; exits non-zero on failure	none	\$9.6
<code>ingest/probe_heading.py</code>	Why a heading did or did not classify	<code>--page N,N</code> , <code>--min-page</code> , <code>--all-blocks</code> , <code>--generic</code>	\$10.1
<code>services/retrieval_svc.py</code>	BGE-M3 encode + reranker score, port 8081	env: <code>RETRIEVAL_DEVICE</code> , <code>EMBED_MAX_LEN</code>	\$8.4
<code>services/rag_api.py</code>	Orchestration, gate, verifier, UI route, port 8080	env: <code>TAU_ABSTAIN</code> , <code>TAU_INCLUDE</code> , <code>RAG_MODEL</code> , <code>RAG_STATIC</code>	\$11, \$13
<code>services/prompts/answer.py</code>	The eight-rule system prompt	—	\$12
<code>services/static/index.html</code>	Web UI, served at /	—	\$13
<code>services/Modelfile</code>	Ollama serving profile: 16k context, temp 0.1, seed 42	—	\$7.3
<code>eval/xml_coverage.py</code>	Indexed clauses vs an independent reference	<code>--doc-id</code> , <code>--show</code>	\$9.7
<code>eval/run_eval.py</code>	Golden set harness; coloured per-question verdict	<code>--answers</code> , <code>--only</code> , <code>--self-test</code> , <code>--no-color</code>	\$15.2
<code>requirements.txt</code>	Pinned dependencies	—	\$8.2

Files you write yourself: `corpus/registry.yaml` and `eval/golden.yaml`.

Chapter 22

Appendix E — Install checklist

[]	1. nvidia-smi: Quadro RTX 5000, 16384 MiB, CUDA >= 12.1	(5.1)
[]	2. Python 3.11, venv at C:\rag\venv	(5.2)
[]	3. HF_HOME set BEFORE any model download, verified in new shell	(5.3)
[]	4. Tesseract + language packs, TESSDATA_PREFIX set	(5.4)
[]	5. .wslconfig caps WSL2 at 12 GB; wsl --shutdown applied	(5.5)
[]	6. Defender exclusion for C:\rag agreed with IT	(5.6)
[]	7. docker volume create qdrant-storage	(6.1)
[]	8. compose up; healthz passes; API key stored	(6.3)
[]	9. docker compose port shows 127.0.0.1, NOT 0.0.0.0	(6.3)
[]	10. docker inspect confirms named volume + snapshot bind mount	(6.3)
[]	11. down/up cycle: collections survive	(6.3)
[]	12. Ollama env set: flash attn, KV q8_0, keep-alive -1	(7.2)
[]	13. std-rag created from Modelfile and responds	(7.3)
[]	14. ollama ps reports 100% GPU	(7.3)
[]	15. torch.cuda.get_device_capability() returns (7, 5)	(8.1)
[]	16. requirements.txt installed, versions pinned	(8.2)
[]	17. Model revisions recorded from scan-cache	(8.3)
[]	18. retrieval-svc /rerank separates a match from a non-match	(8.4)
[]	19. registry.yaml written; page_offset VERIFIED, not assumed	(9.1)
[]	20. triage.py: text layer present on every document	(9.2)
[]	21. parse.py self-test passes; headings look right on the PDF	(9.3)
[]	22. index.py --dry-run: no collision, clause/page within target	(9.4)
[]	23. Ingest completed; points_count matches chunks written	(9.5)
[]	24. verify_ingest.py meets all four criteria	(9.6)
[]	25. Coverage measured against an independent reference	(9.7)
[]	26. THREE citations spot-checked by hand against the PDF	(9.8)
[]	27. TAU_ABSTAIN and TAU_INCLUDE understood, not just inherited	(11.2)
[]	28. answer_contract.txt in place, rules 3, 6 and 7 verbatim	(12)
[]	29. Web UI reachable at http://127.0.0.1:8080	(13)
[]	30. Positive smoke test returns a cited answer	(13)
[]	31. NEGATIVE smoke test returns status not_in_corpus	(13)
[]	32. Both services under NSSM; reboot test passes	(14.4)
[]	33. golden.yaml written, >= 40 questions, composition per 15.1	(15.1)
[]	34. run_eval.py passes all gates, ZERO fabrications	(15.3)
[]	35. Snapshot taken and restore tested	(16.1)
[]	36. Licence and IP register completed and signed	(App. C)
[]	37. Version pin table filled in with actual versions	(App. A)

Items 26, 31, 34 and 36 are the gates. The system may be technically complete without them and is not fit for the stated use case until they pass.

Chapter 23

Appendix F — Sources consulted

Model and licence facts in chapter 2 were verified against current sources rather than recalled.

Mistral Small 4 architecture, release date, memory requirements

freeainews.com /open-source/mistral-small-4-apache-open-source-2026/
aizolo.com /blog/mistral-ai-latest-models-2026/
openrouter.ai /mistralai/mistral-small-2603

Ministral 3 14B capabilities, licence, context window, VRAM

huggingface.co /mistralai/Ministral-3-14B-Instruct-2512-GGUF
huggingface.co /unsloth/Ministral-3-14B-Instruct-2512-GGUF

BGE-M3 retrieval modes, language coverage, long-context behaviour,
and the BGE-M3 + BGE-reranker-v2 production pairing

checkthat.ai /answers/what-are-the-best-embedding-models

bge-reranker-v2-m3 characterisation and comparative results

arXiv 2606.25249 adaptive re-ranking
arXiv 2607.04043 CheckThat! 2026 multilingual claim-source retrieval

Qwen3-Reranker comparative scores (MTEB-R, MMTEB-R)

arXiv 2506.05176 Qwen3 Embedding technical report

PyMuPDF dual AGPL-3.0 / commercial licensing

pypi.org /project/PyMuPDF/

EASA Easy Access Rules and CS-25 publication formats

easa.europa.eu /en/document-library/easy-access-rules

Where sources disagree — notably on reranker ordering — both are cited in the body, and the disagreement is resolved by measurement on your own golden set rather than by preferring one leaderboard.